



Microsoft Purview DLP Connector — Credential Provisioning Guide

Version 0.2.0, 2026-07-16

Table of Contents

Overview.....	1
What kind of account to create.....	1
Permissions to grant	1
Generating the credential.....	2
Mapping values into Crogl.....	2
Verifying the setup.....	3
Troubleshooting: "Account is not provisioned".....	3

Overview

The Crogl Purview DLP connector reads Data Loss Prevention alerts from Microsoft Graph's unified security alerts surface (`/v1.0/security/alerts_v2`, filtered to `serviceSource eq 'dataLossPrevention'`). It authenticates as an application, not as a signed-in user, using the OAuth2 client-credentials grant—so there is no interactive login step and no per-analyst account. Everything in this guide happens once, in the customer's own Microsoft Entra ID tenant, before the connector is configured in Crogl.

These steps require **Global Administrator** or **Application Administrator** access in the customer's tenant, plus a role that can grant admin consent (Global Administrator or Privileged Role Administrator). If you're an FDE walking a customer through this live, expect their tenant admin to drive the Entra admin center while you confirm each value.

NOTE

This connector only reads DLP alerts. It does not touch Purview's Audit, eDiscovery, or Data Map surfaces—those would need separate app permissions and, if ever built, a separate Crogl connector.

What kind of account to create

Create an **app registration** (a service principal), not a user account.

1. Sign in to the [Microsoft Entra admin center](#) as a Global Administrator or Application Administrator.
2. Go to **Identity > Applications > App registrations > New registration**.
3. Name it something identifiable, e.g. **Crogl Purview DLP Connector**.
4. Supported account type: **Single tenant** (accounts in this organizational directory only).
5. Leave **Redirect URI** blank—the client-credentials grant is unattended and never redirects a browser.
6. Select **Register**.

After registration, the app's **Overview** page shows two of the three values this guide ends with: the **Application (client) ID** and the **Directory (tenant) ID**.

Permissions to grant

1. In the app registration, go to **API permissions > Add a permission > Microsoft Graph**.
2. On the permission-type screen, select the **Application permissions** tile—not Delegated. There is no signed-in user in this flow (client-credentials grant), so a Delegated grant will never work here, no matter what's added under it.
3. Search for **SecurityAlert.Read.All** and select it.
4. Select **Add permissions**.
5. Back on the API permissions page, select **Grant admin consent for <tenant name>** and

confirm.

WARNING

`SecurityAlert.Read.All` exists as **two distinct permissions with the identical name**—one under the Delegated tile, one under Application. Entra's UI does not disambiguate them beyond which tile you picked before searching. It's entirely possible to add the Delegated one, grant admin consent on it, and see everything show a green "Granted" status—and the connector will still fail with **403** at query time, because a client-credentials token can never satisfy a Delegated grant.

The reliable check afterward: on the **API permissions** page, the **Type** column next to `SecurityAlert.Read.All` must read **Application**, not **Delegated**. If it reads Delegated, remove it and re-add it from the Application permissions tile.

The permission shows a status of **Granted** once consent succeeds. Skipping admin consent is the second most common setup mistake here—Graph returns **HTTP 403** on every query until it's granted, even when the (correct, Application-type) permission is already added.

NOTE

This connector only reads `alerts_v2`, filtered to the DLP service source. If a future resource needs a different Graph permission (for example, `SecurityAlert.ReadWrite.All` to update alert status), add it here and re-grant consent—don't over-grant permissions the connector doesn't use.

Generating the credential

1. In the app registration, go to **Certificates & secrets > Client secrets > New client secret**.
2. Add a description (e.g. `crogl-connector`) and an expiry consistent with the customer's credential-rotation policy.
3. Select **Add**.
4. **Copy the secret's Value immediately**—Entra shows it once, at creation time, and never again. If it's missed, delete the secret and generate a new one; there is no way to retrieve a lost value.

Mapping values into Crogl

Three values from the steps above map directly to `community/purview-dlp/connector.json`'s `'credential_schema.fields` when creating the connector instance in Crogl:

Crogl field	Where it came from	Notes
<code>tenant_id</code>	App registration Overview > Directory (tenant) ID	Identifies the customer's Entra tenant.
<code>client_id</code>	App registration Overview > Application (client) ID	Identifies this specific app registration.
<code>client_secret</code>	Certificates & secrets > the secret Value copied at creation	Treat as a password. Never paste it into chat, a ticket, or this document.

Crogl exchanges these server-side for a bearer token via `https://login.microsoftonline.com/<tenant_id>/oauth2/v2.0/token` (client-credentials grant, `https://graph.microsoft.com/.default` scope) and attaches it automatically on every Graph call — no further action needed once the three values are entered.

Verifying the setup

Before handing credentials to Crogl, or right after creating the connector instance, confirm the setup is correct:

- **In Entra:** API permissions shows `SecurityAlert.Read.All` with **Type** = **Application** (not Delegated) and **Status** = **Granted for <tenant>**.
- **In Crogl:** run a query against the connector (e.g. `purview_dlp.py query --filter "" --limit 1`) and confirm an `HTTP 200` response.
- **If you get 401:** the client ID, tenant ID, or secret value is wrong, or the secret has expired — re-check each value against the app registration.
- **If you get 403 with message text like "Insufficient privileges...":** check the **Type** column first — this is almost always the Delegated-vs-Application mix-up above, not a consent problem. If Type already reads Application, admin consent likely wasn't granted — revisit [Permissions to grant](#).
- **If you get 403 with "code":"Unauthorized" and message "Account is not provisioned":** this is a different problem from the one above, see [Troubleshooting: "Account is not provisioned"](#) — don't retry the same permission-grant steps, they're already correct if you're seeing this specific message.
- **Zero rows back is expected** on a tenant with no DLP alerts in the retention window yet, or one where Purview DLP policies aren't configured — that's a successful, empty query, not an error.

NOTE

DLP alerts only appear at all once the customer has at least one active Microsoft Purview DLP policy configured (Exchange, SharePoint/OneDrive, or Endpoint) that has matched content, **and** that policy's matching rule has "Send an alert to admins when a rule match occurs" turned on. This is not automatic — Microsoft's own default policies (e.g. "Default Office 365 DLP policy") ship with this off, notify-only, no alert generated, so a customer can have DLP fully licensed, configured, and actively matching content and still see zero rows here indefinitely. Check the rule's Actions > Incident reports section, not just whether the policy itself is "On".

Troubleshooting: "Account is not provisioned"

Confirmed live against an internal test tenant (2026-07-15/16): a `403` with `"code":"Unauthorized","message":"Unauthorized request - Account is not provisioned."` on `/security/alerts_v2` can persist even with everything above done correctly — `SecurityAlert.Read.All` granted as Application type, admin consent confirmed, a real

client secret verified via a direct token-mint test, and a genuine DLP policy match with alert-generation turned on. The same error also reproduces identically on `/security/incidents`, but not on the legacy `/security/alerts` endpoint (which fails differently, on permissions, since it needs a different scope)—so this is specific to the modern `microsoft.graph.security` alerts-and-incidents surface, not a permissions problem and not `alerts_v2`-specific.

This points to the tenant's underlying Microsoft Defender unified alerts/incidents data plane not being provisioned, separate from whether Purview DLP itself is licensed and configured. Two things to check, in order:

1. **Visit the Microsoft Defender portal at least once**, signed in as an admin. If the tenant has never been visited there, this can trigger onboarding. In our test case this alone did not resolve it—the portal loaded a normal, already-active dashboard, so this tenant had already been onboarded at the portal level.
2. **Confirm the Unified Audit Log is turned on**: Purview > Explorers > Activity explorer will show a banner ("To use this feature, turn on auditing...") if it's off. This is a distinct, separate toggle from Purview DLP licensing or policy configuration, and in our test case, it was off. Once turned on, Microsoft's own guidance is that it can take **up to 24 hours** to fully propagate before the unified alerts data plane (and therefore this connector) starts working.

If both of the above are already correct and the error still persists after a full 24-hour window, this is likely a genuine Microsoft-side tenant provisioning gap rather than anything fixable from the customer's admin console — escalate to a Microsoft support case rather than continuing to re-check permissions.